rails7/e1/ar/transactions.rb

```
peter = Account.create(balance: 100, number: "12345")
paul  = Account.create(balance: 200, number: "54321")

Account.transaction do
  paul.deposit(10)
  peter.withdraw(10)
end
```

We check the database, and, sure enough, the money got transferred:

```
depot> sqlite3 -line db/development.sqlite3 "select * from accounts"
     id = 1
 number = 12345
balance = 90

     id = 2
 number = 54321
balance = 210
```

Now let's get radical. If we start again but this time try to transfer $350, we'll run Peter into the red, which isn't allowed by the validation rule. Let's try it:

rails7/e1/ar/transactions.rb

```
peter = Account.create(balance: 100, number: "12345")
paul  = Account.create(balance: 200, number: "54321")
```

rails7/e1/ar/transactions.rb

```
Account.transaction do
  paul.deposit(350)
  peter.withdraw(350)
end
```

When we run this, we get an exception reported on the console:

```
.../validations.rb:736:in `save!': Validation failed: Balance is negative
from transactions.rb:46:in `adjust_balance_and_save!'
  :        :        :
from transactions.rb:80
```

Looking in the database, we can see that the data remains unchanged: